

The PERFORM

The **PERFORM** Verb

Iteration is an important programming construct. We use iteration when we need to repeat the same instructions over and over again.

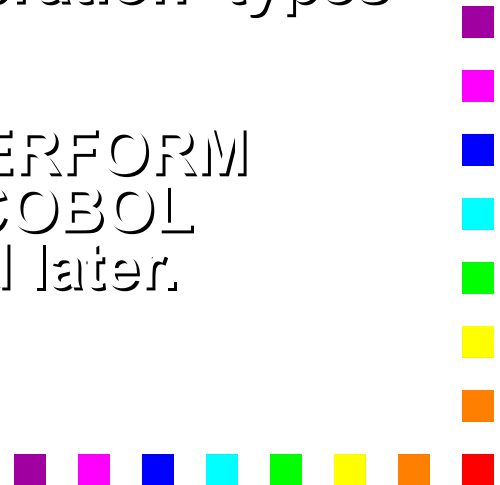
Most programming languages have several iteration keywords (e.g. WHILE, FOR, REPEAT) which facilitate the creation different 'types' of iteration structure.

COBOL only has **one** iteration construct; **PERFORM**.

But the **PERFORM** has several variations.

Each variation is equivalent to one of the iteration 'types' available in other languages.

This lecture concentrates on three of the **PERFORM** formats. The **PERFORM VARYING**, the COBOL equivalent of the FOR, will be introduced later.



Paragraphs :- Revisited

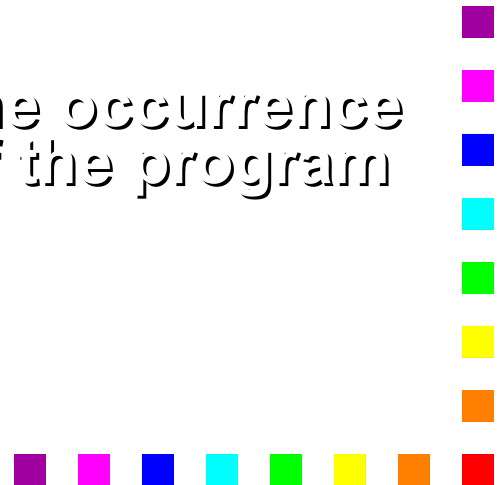
A Paragraph is a block of code to which we have given a name.

A Paragraph Name is a programmer defined name formed using the standard rules for programmer defined names (A-Z, 0-9, -).

A Paragraph Name is **ALWAYS** terminated with a 'full-stop'.

Any number of statements and sentences may be included in a paragraph, and the last one (at least) must be terminated with a 'full-stop'.

The scope of a paragraph is delimited by the occurrence of another paragraph name or the end of the program text.



Paragraph Example

ProcessRecord.

DISPLAY StudentRecord

READ StudentFile

AT END MOVE HIGH-VALUES TO StudentRecord

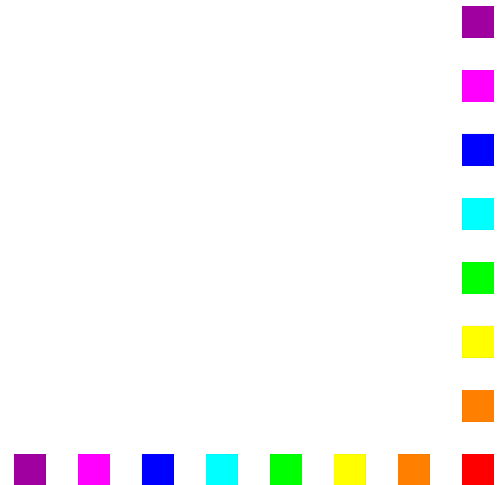
END-READ.

ProduceOutput.

DISPLAY "Here is a message".

NOTE

The scope of 'ProcessRecord' is delimited by the occurrence the paragraph name 'ProduceOutput'.



Format 1 Syntax.

$$\underline{\text{PERFORM}} \left[\text{1stProc} \left[\left\{ \begin{array}{c} \underline{\text{THRU}} \\ \underline{\text{THROUGH}} \end{array} \right\} \text{EndProc} \right] \right]$$

This is the only type of PERFORM that is **not** an iteration construct.

It instructs the computer to **transfer control** to an out-of-line block of code.

When the end of the block is reached, control reverts to the statement (not the sentence) immediately following the PERFORM.

1stProc and EndProc are the names of Paragraphs or Sections.

The PERFORM.THROUGH instructs the computer to treat the Paragraphs or Sections from 1stProc TO EndProc as a single block of code.



Format 1 Example.

Run of PerformFormat1

```
In TopLevel. Starting to run program
>>>> Now in OneLevelDown
>>>>>>> Now in TwoLevelsDown.
>>>> Back in OneLevelDown
Back in TopLevel.
```

PROCEDURE DIVISION.

TopLevel.

DISPLAY "In TopLevel. Starting to run program"

PERFORM OneLevelDown

DISPLAY "Back in TopLevel.".

STOP RUN.

TwoLevelsDown.

DISPLAY ">>>>>>> Now in TwoLevelsDown."

OneLevelDown.

DISPLAY ">>>> Now in OneLevelDown"

PERFORM TwoLevelsDown

DISPLAY ">>>> Back in OneLevelDown".

Format 1 Example.

Run of PerformFormat1

```
In TopLevel. Starting to run program
>>>> Now in OneLevelDown
>>>>>>> Now in TwoLevelsDown.
>>>> Back in OneLevelDown
Back in TopLevel.
```

PROCEDURE DIVISION.

TopLevel.

```
    DISPLAY "In TopLevel. Starting to run program"
    PERFORM OneLevelDown
    DISPLAY "Back in TopLevel.".
    STOP RUN.
```

TwoLevelsDown.

```
    DISPLAY ">>>>>>> Now in TwoLevelsDown."
```

OneLevelDown.

```
    DISPLAY ">>>> Now in OneLevelDown"
    PERFORM TwoLevelsDown
    DISPLAY ">>>> Back in OneLevelDown".
```

Format 1 Example.

Run of PerformFormat1

```
In TopLevel. Starting to run program
>>>> Now in OneLevelDown
>>>>>>> Now in TwoLevelsDown.
>>>> Back in OneLevelDown
Back in TopLevel.
```

PROCEDURE DIVISION.

TopLevel.

```
    DISPLAY "In TopLevel. Starting to run program"
    PERFORM OneLevelDown
    DISPLAY "Back in TopLevel.".
    STOP RUN.
```

TwoLevelsDown.

```
    DISPLAY ">>>>>>> Now in TwoLevelsDown."
```

OneLevelDown.

```
    DISPLAY ">>>> Now in OneLevelDown"
    PERFORM TwoLevelsDown
    DISPLAY ">>>> Back in OneLevelDown".
```


Format 1 Example.

Run of PerformFormat1

```
In TopLevel. Starting to run program
>>>> Now in OneLevelDown
>>>>>>> Now in TwoLevelsDown.
>>>> Back in OneLevelDown
Back in TopLevel.
```

PROCEDURE DIVISION.

TopLevel.

```
    DISPLAY "In TopLevel. Starting to run program"
    PERFORM OneLevelDown
    DISPLAY "Back in TopLevel.".
    STOP RUN.
```

TwoLevelsDown.

```
    DISPLAY ">>>>>>> Now in TwoLevelsDown."
```

OneLevelDown.

```
    DISPLAY ">>>> Now in OneLevelDown"
```

```
    PERFORM TwoLevelsDown
```

```
    DISPLAY ">>>> Back in OneLevelDown".
```

Format 1 Example.

Run of PerformFormat1

```
In TopLevel. Starting to run program
>>>> Now in OneLevelDown
>>>>>>>> Now in TwoLevelsDown.
>>>> Back in OneLevelDown
Back in TopLevel.
```

PROCEDURE DIVISION.

TopLevel.

```
    DISPLAY "In TopLevel. Starting to run program"
    PERFORM OneLevelDown
    DISPLAY "Back in TopLevel.".
    STOP RUN.
```

TwoLevelsDown.

```
    DISPLAY ">>>>>>>> Now in TwoLevelsDown."
```

OneLevelDown.

```
    DISPLAY ">>>> Now in OneLevelDown"
    PERFORM TwoLevelsDown
    DISPLAY ">>>> Back in OneLevelDown".
```

Format 1 Example.

Run of PerformFormat1

```
In TopLevel. Starting to run program
>>>> Now in OneLevelDown
>>>>>>> Now in TwoLevelsDown.
>>>> Back in OneLevelDown
Back in TopLevel.
```

PROCEDURE DIVISION.

TopLevel.

```
    DISPLAY "In TopLevel. Starting to run program"
    PERFORM OneLevelDown
    DISPLAY "Back in TopLevel.".
    STOP RUN.
```

TwoLevelsDown.

```
    DISPLAY ">>>>>>> Now in TwoLevelsDown."
```

OneLevelDown.

```
    DISPLAY ">>>> Now in OneLevelDown"
    PERFORM TwoLevelsDown
    DISPLAY ">>>> Back in OneLevelDown".
```

Format 1 Example.

Run of PerformFormat1

```
In TopLevel. Starting to run program
>>>> Now in OneLevelDown
>>>>>>> Now in TwoLevelsDown.
>>>> Back in OneLevelDown
Back in TopLevel.
```

PROCEDURE DIVISION.

TopLevel.

```
    DISPLAY "In TopLevel. Starting to run program"
    PERFORM OneLevelDown
    DISPLAY "Back in TopLevel."
    STOP RUN.
```

TwoLevelsDown.

```
    DISPLAY ">>>>>>> Now in TwoLevelsDown."
```

OneLevelDown.

```
    DISPLAY ">>>> Now in OneLevelDown"
    PERFORM TwoLevelsDown
    DISPLAY ">>>> Back in OneLevelDown".
```

Why use the PERFORM Thru?

PROCEDURE DIVISION.

Begin.

PERFORM SumSales

STOP RUN.

SumSales.

Statements

Statements

IF NoErrorFound

Statements

Statements

IF NoErrorFound

Statements

Statements

Statements

END-IF

END-IF.

Go To and PERFORM THRU

PROCEDURE DIVISION

Begin.

PERFORM SumSales THRU SumSalesExit
STOP RUN.

SumSales.

Statements

Statements

IF ErrorFound GO TO SumSalesExit

END-IF

Statements

Statements

Statements

IF ErrorFound GO TO SumSalesExit

END-IF

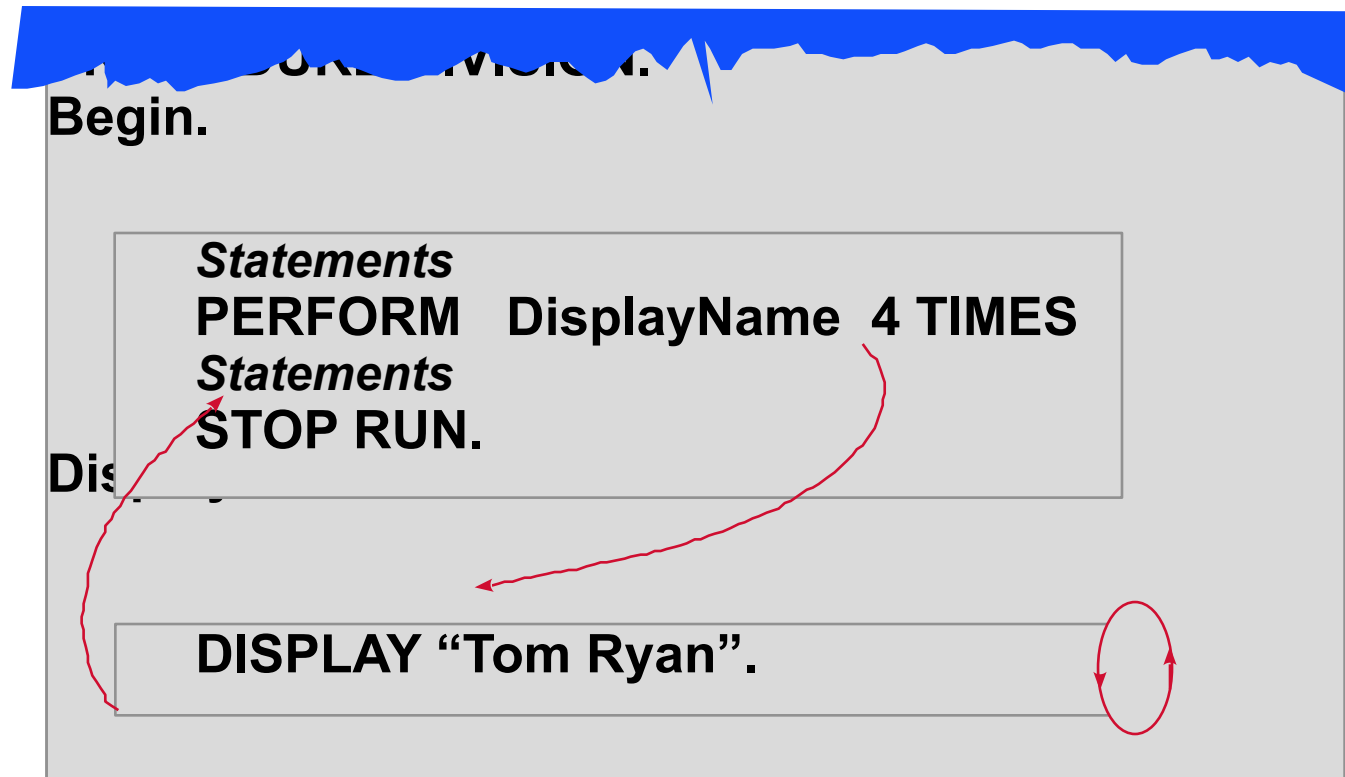
Statements

SumSalesExit.

EXIT.

Format 2 - Syntax

PERFORM $\left[\text{1stProc} \left\{ \begin{array}{c} \text{THRU} \\ \text{THROUGH} \end{array} \right\} \text{EndProc} \right]$
RepeatCount TIMES
[StatementBlock END - PERFORM]



Format 2 Example

Run of PerformExample2

```
$ SET SOURCEFORMAT"FREE"  
IDENTIFICATION DIVISION.  
PROGRAM-ID.    PerformExample2.  
AUTHOR.    Michael Coughlan.
```

```
DATA DIVISION.  
WORKING-STORAGE SECTION.  
01 NumofTimes          PIC 9 VALUE 5.
```

```
PROCEDURE DIVISION.
```

```
Begin.
```

```
    DISPLAY "Starting to run program"
```

```
    PERFORM 3 TIMES
```

```
        DISPLAY ">>>>This is an in line Perform"
```

```
    END-PERFORM
```

```
    DISPLAY "Finished in line Perform"
```

```
    PERFORM OutOfLineEG NumOfTimes TIMES
```

```
    DISPLAY "Back in Begin. About to Stop".
```

```
    STOP RUN.
```

```
OutOfLineEG.
```

```
    DISPLAY ">>>> This is an out of line Perform".
```

Starting to run program

>>>>This is an in line Perform

>>>>This is an in line Perform

>>>>This is an in line Perform

Finished in line Perform

>>>> This is an out of line Perform

>>>> This is an out of line Perform

>>>> This is an out of line Perform

>>>> This is an out of line Perform

>>>> This is an out of line Perform

Back in Begin. About to Stop

Format 3 Syntax

PERFORM $\left[\text{1stProc} \left[\left\{ \begin{array}{c} \text{THRU} \\ \text{THROUGH} \end{array} \right\} \text{EndProc} \right] \right] \left[\text{WITH } \text{TEST} \left\{ \begin{array}{c} \text{BEFORE} \\ \text{AFTER} \end{array} \right\} \right]$
UNTIL Condition
[StatementBlock END - PERFORM]

This format is used where the **WHILE** or **REPEAT** constructs are used in other languages.

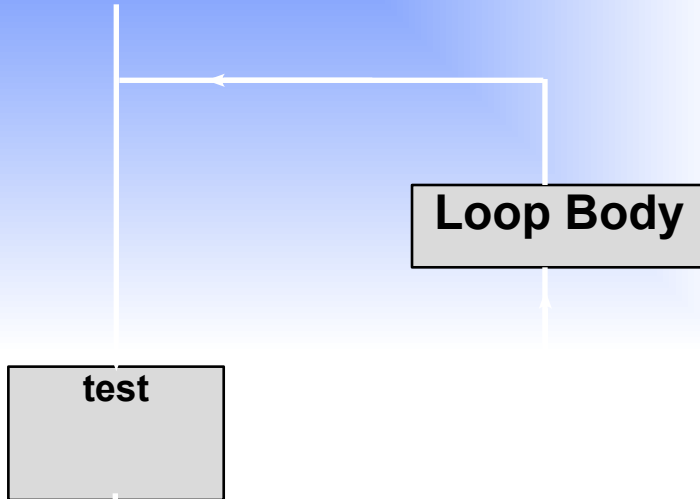
If the **WITH TEST BEFORE** phrase is used the PERFORM behaves like a **WHILE** loop and the condition is tested **before** the loop body is entered.

If the **WITH TEST AFTER** phrase is used the PERFORM behaves like a **REPEAT** loop and the condition is tested **after** the loop body is entered.

The **WITH TEST BEFORE** phrase is the **default** and so is rarely explicitly stated.



**PERFORM WITH
TEST BEFORE =
WHILE ... DO**



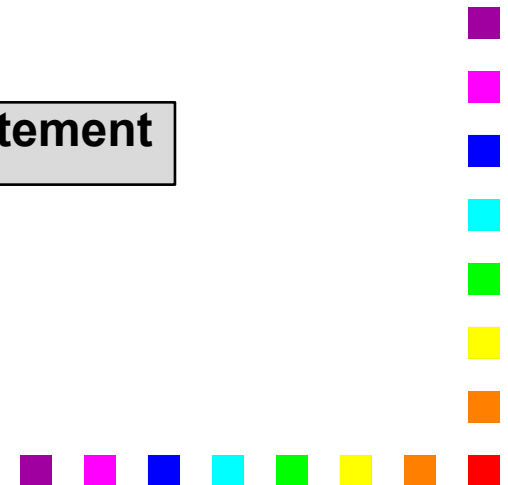
Next Statement

**PERFORM WITH
TEST AFTER =
REPEAT ... UNTIL**

Loop Body

test

Next Statement



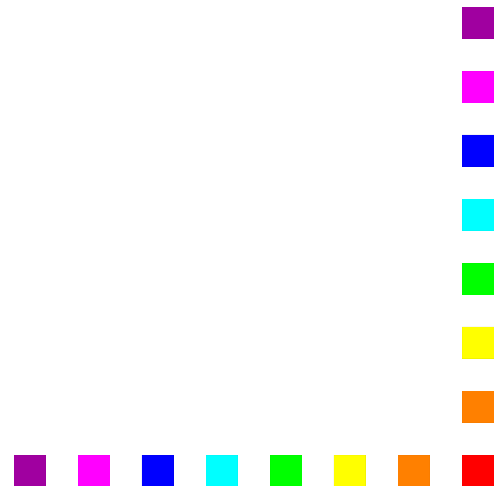
Sequential File Processing

In general terms, the **WHILE** loop is an ideal construct for processing sequences of data items whose length is not predefined.

Such sequences of values are often called “streams”.

Because the ‘length’ of the stream is unknown we have to be careful how we manage the detection of the end of the stream.

A useful way for solving this problem uses a strategy known as “read ahead”.



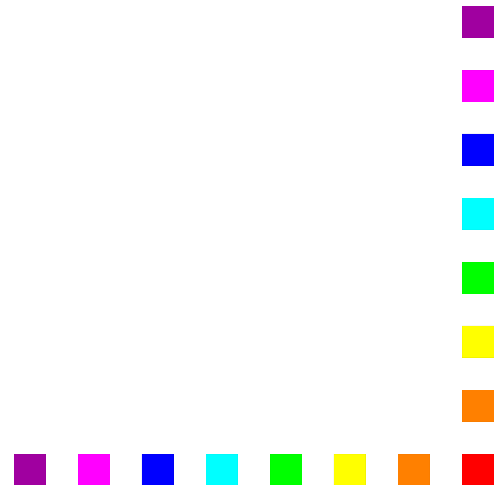
The READ Ahead

With the “read ahead” strategy we always **try** to stay one data item ahead of the processing.

The general format of the “read ahead” algorithm is as follows;

```
Attempt to READ first data item
WHILE NOT EndOfStream
    Process data item
    Attempt to READ next data item
ENDWHILE
```

Use this to process any stream of data.



Reading a Sequential File

Algorithm Template

READ StudentRecords

AT END MOVE HIGH-VALUES TO StudentRecord

END-READ

PERFORM UNTIL StudentRecord = HIGH-VALUES

DISPLAY StudentRecord

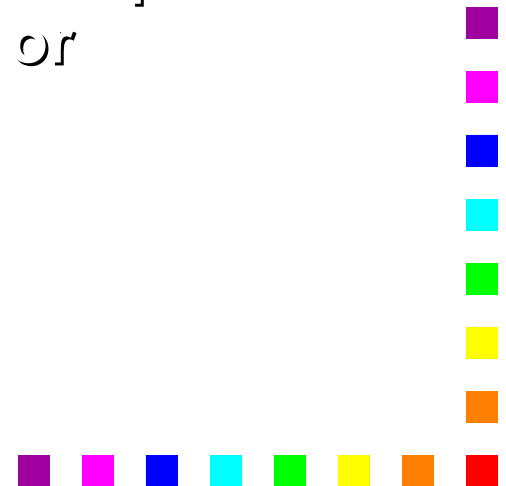
READ StudentRecords

AT END MOVE HIGH-VALUES TO StudentRecord

END-READ

END-PERFORM

This is an example of an algorithm which is capable of processing any sequential file; ordered or unordered!



RUN OF SeqRead

9456789 COUGHLANMS LM51

9367892 RYAN TG LM60

9368934 WILSON HR LM61

PROCEDURE DIVISION.

Begin.

OPEN INPUT StudentFile

READ StudentFile

AT END MOVE HIGH-VALUES TO StudentDetails

END-READ

PERFORM UNTIL StudentDetails = HIGH-VALUES

DISPLAY StudentId SPACE StudentName SPACE CourseCode

READ StudentFile

AT END MOVE HIGH-VALUES TO StudentDetails

END-READ

END-PERFORM

CLOSE StudentFile

STOP RUN.